Homework – Get 1 Alphabetic Character

This is a programming assignment that will be graded and scored as a homework since it is so simple. This one is less simple than previous assignments.

Write a program for the LC3. This program should be named ***hwGet1Alpha.asm.***

The "MAIN" part of your program (the top few lines) should look like this. NOTE that this will run your subroutine TWICE and therefore will get TWO characters.

```
                .orig x3000
                LEA R0, PROMPT
                PUTS
                JSR  GETALPHA ; EACH CALL TO GETALPHA GETS ONE CHARACTER
                JSR  GETALPHA ;Calling it again gets a second character.
                HALT
PROMPT          .stringz "Enter two letters: "  ; Note there MUST be a single space after the :

;Your GETALPHA routine should follow.
```

GETALPHA should do the following:

Get a single alphabetic character from the user using GETC. Note that GETC is a system subroutine and will affect R7. That is, if you are in a subroutine (and you are) and you call GETC, it will save its return address in R7 and wipe out your subroutine's return address.

If the character typed is the newline character (ASCII 10). If the enter key is pressed during a call to GETC the value 10 will be placed into R0. If enter is pressed the program SHOULD NOT print the newline but should simply return with the 10 in R0.

If the character typed is the underscore character (ASCII 95), print it and return.

If the character typed is an alphabetic character (a-z or A-Z), display the character and return with that characters ASCII code in R0.

If any other character is typed, the routine should not print anything, but should allow the user to enter a new character.

The call to the subroutine should not affect any Registers other than R0.

Running your program should print the prompt. **NOTE: There IS a single space after the : character.**

```
Enter two letters:
```

The program should stop and allow you to type a key. It should ONLY allow a-z, A-Z, or newline. Any other character should be ignored and not printed.

Once a letter is pressed you should see the letter appear after the prompt that was previously printed. For example, if the A key is pressed you should see:

```
Enter two letters: A
```

After you press the second character, it should then appear after the A. For example, if B were pressed next you would see the following. After the second character is entered the program should end

```
Enter two letters: AB
```

If you run the program and press an _ you should see it. If you press _ followed by A you should see:

```
Enter two letters: _A
```

Also note the newline returns but does not print anything. If you run the program again and press enter followed by A you should see the following and the program would end.

```
Enter two letters: A
```

Submit your ***hwGet1Alpha.asm*** file to Web-CAT when done. Make sure to test your program with characters OTHER than a-z, A-Z, newline, and underscore. Make especially sure to test with the characters immediately before 'a' and 'A' and immediately after 'z' and 'Z'. Make sure to test the enter key (newline) and the underscore character.

On the next page is an English outline for the GETALPHA subroutine.

GETALPHA

1. **SAVE REGISTERS** (Do not save or restore R0 since it will be returning something.)
2. **GET A CHARACTER** into R0 (This uses a trap, so you will need to save R7 also)
3. If the character is a newline (R0==10), we are done. Branch to RESTORE REGISTERS.
4. If the character is an underscore (R0==95), we are done. Branch to PRINT CHARACTER
5. If the code in R0 comes before the 'A' character, we know it is a bad character, so branch back to GET A CHARACTER
6. If the code in R0 comes before 'Z' or is the 'Z' character, we know it is correct so branch to PRINT CHARACTER. (We know from the previous if that the character is 'A' or later, so if we ALSO know it comes before 'Z' or is 'Z' that the character must be in the range 'A' to 'Z'.)
7. If the code in R0 comes before 'a', we know it is bad, so branch back to GET A CHARACTER. (The previous two ifs check everything before 'Z', so those ifs eliminated everything up to 'Z'. Anything that passed those checks must be greater than 'Z'. This if checks to see if it is before 'a'. If it is before 'a' it means the character must be one of the characters between 'Z' and 'a' and is bad).
8. If the code in R0 comes before 'z' or is 'z', we know it is correct, so branch to PRINT CHARACTER. (The previous three ifs eliminated everything before 'a'. So if this character is greater than or equal to 'a' and is also, because of this if, less than or equal to 'z', it means the character IS in the range 'a' through 'z' and is a good character.
9. At this point we have checked ALL the allowed characters. If the character in R0 was not one of those, we need to go get another character. So do a BRANCH ALWAYS here to GET A CHARACTER.
10. **PRINT CHARACTER**
11. **RESTORE REGISTERS**. Restore ALL that you saved above including R7.
12. **RETURN**
13. MAKE SURE TO PUT ANY DATA YOU NEED (constants or save space for registers) FOR THE ABOVE HERE AFTER THE RETURN FOR THE SUBROUTINE

How to troubleshoot this program.

Use the "Step Into" button until you get into the GETALPHA subroutine. From this point on, use the "Step Over" button so you don't get stuck going into the GETC code or OUT code.

Click through the code to check for newline. It should be something like this.

```
GETC ;
ADD R1, R0, #-10 ; You don't need to load 10 into a register
                 ; because you can just use the immediate value.
BRZ END ;   END should be where the register restore starts.
```

Click step over on GETC to get a character. Press the ENTER key.

Click step over on ADD to add R0 and -10. Note that R1 should now contain a 0 and the Z flag should be a 1. If it isn't try to figure out why. Something isn't right if you didn't get 0.

Click step over on the BRZ. Your program should branch to the loading of registers at the end of the subroutine. If it does not branch, something isn't right. Figure out why you didn't branch.

If it works, you have successfully tested the Newline entry.

Next you should check for an underscore.

```
GETC ;
ADD R1, R0, #-10 ; You don't need to load 10 because you can just use
the immediate value.
BRZ END ;   END should be where the register restore starts.
LD R1, UNDERSCORE ; Put the negative of the ASCII code for the
                  ; underscore in R1
ADD R1, R0, R1    ; See if R0 contains the underscore.
BRZ PRINT         ; You should have an OUT right before
                  ; restoring registers.  Branch to that OUT.
```

Click on step over on the GETC. Press the underscore character. See the ASCII code in R0.
Click step over for subtracting 10. The value in R1 should not be zero.
Click step over for the first BRZ. It should not branch. If it does something is wrong here.
Click stepover on the LD. You should see the negative value of an underscore in R1.
Check to see that R0 contains the ASCII code for underscore and that R1 contains the negative value of the ASCII code for underscore. If that isn't the case you have loaded or entered the wrong data.
Click step over on the ADD and R1 should be 0 and the Z flag should be set.
Click step over and the BRZ should branch to OUT and it will print the underscore, restore the registers, and